

Backend Design Task: Webhook Ingestion

Context

TravelCard issues corporate cards through an external card issuer (Cal). Cal pushes card lifecycle events (`activated`, `blocked`, `transaction_declined`, `replaced`, `expired`, ...) to our backend via a webhook. Each event triggers downstream side effects: a push notification to the cardholder, an in-app message, an audit entry, and a state update in MongoDB.

The webhook lives on a public endpoint. Our backend runs as multiple ECS Fargate tasks behind an Application Load Balancer (3+ instances in steady state, more during autoscaling).

What we want from you

A short design document describing how you would build `POST /webhooks/cal/card-status`. **No code.** We want to see how you think about the system.

Time expectation: ~2 hours. If you find yourself going significantly beyond that, stop and submit what you have. We'll cover the rest in the live discussion.

Deliverables

1. **Architecture diagram** — components, data flow, and the boundaries between them. Excalidraw, Lucid, draw.io, whiteboard photo — any format is fine.
2. **Written design (~1.5 pages)** — walking through the diagram and addressing the four questions below.
3. **A short "what I'd do differently with more time/budget" paragraph** — what you deliberately cut, and why.

We'll discuss your design live afterward, including topics not covered in the writeup (poison events, cost at scale, failure modes, alternatives you considered).

Sample event

json

```
JSON
{
  "event_id": "evt_01HZ7K3M9XQ8YBN2VFRT4PWCAE",
  "event_type": "card.blocked",
  "card_id": "card_abc123",
  "user_id": "usr_xyz789",
```

```
"occurred_at": "2026-05-19T08:23:11Z",  
"reason": "lost_or_stolen",  
"metadata": {}  
}
```

For each event, the system must:

- Persist the new card state.
- Send a push notification to the cardholder.
- Write an audit entry.

Operating conditions (read carefully — these are the real requirements)

- The endpoint is reachable from the public internet. Anyone can curl it.
- Cal will retry on non-2xx responses. They also occasionally retry on 2xx — their docs are vague and we've observed it in staging.
- Cal does not guarantee event ordering.
- Network blips between us and MongoDB / our notification provider happen ~weekly.
- The notification provider is best-effort and rate-limited; sending the same notification twice is a user-visible bug.
- We cannot afford to drop events. A process crash, a DB outage, or a deploy mid-request must not cause an event to vanish silently.
- The endpoint should scale horizontally and be safe to deploy behind autoscaling.

Critical constraint — please address this directly

Multiple backend pods can receive the **same event_id**. The ALB routes any single delivery to exactly one pod, but Cal retries the same event independently, and those retries can land on different pods. A single logical event must produce **exactly one push notification and exactly one audit entry** across all pods and all retries.

Naive "check-then-write" logic will race. The mechanism you pick for adjudicating this race is the single most important decision in your design. Be specific about where the dedup key lives, what operation makes the race safe, and what the losing pod does.

Address each of these in your writeup

We grade on the reasoning, not the specific tech you pick. AWS, GCP, on-prem — your call, as long as you justify it.

1. **Edge / perimeter** — what sits in front of the endpoint, and why. How do you make sure traffic that reaches your app actually came from Cal? What does an attacker see when they hit the endpoint directly?
2. **Sync vs. async + durability** — what does your handler do synchronously before responding 2xx to Cal, and what gets deferred? Walk through what happens to an event if (a) your process crashes after receiving the request but before doing

anything, (b) MongoDB is unreachable for 30 seconds, (c) the push provider is down for 2 hours. At each stage, where does the event live, and how does it eventually get processed?

3. **Exactly-once side effects (the critical constraint above)** — how do you guarantee one notification and one audit entry per event across N concurrent pods? Be concrete about the mechanism: where the dedup key lives, the exact atomic operation that resolves the race, TTL/lifetime of the key, and what wins on a race.
4. **Observability** — what do you log, what do you trace, what do you alert on, and what do you deliberately **not** log? Walk through how you'd debug "the customer says they didn't get a notification 8 hours ago."

Out of scope

- Writing or pseudo-coding the handler.
- A real Cal integration.
- The mobile/web side of receiving the push.
- Detailed IaC.

What we'll discuss live (don't write about these — save them for the conversation)

- Poison events: what happens to an event that fails 10 times in a row, and who finds out.
- Cost and operational footprint at 10 events/sec vs 1000 events/sec; which component breaks first as load grows.
- Failure modes you considered and rejected.
- Alternatives to the tech you picked.